# Public examples: Chinchilla, optimizer grids, and Delphi

> **Reading note.** This is an explanatory reconstruction of Jiaxuan Zou's literature synthesis. Published experiments belong to the cited teams. Numbered protocols and illustrative matrices are our planning synthesis unless explicitly attributed.

The blog's appendix compares published configurations. These examples answer different questions and therefore use different run geometries and tuning policies.

| Example | Central experiment | What transfers as methodology |
|---|---|---|
| Chinchilla | Vary size at fixed compute; identify slice minima | Separate model and token allocation |
| Fantastic Optimizers | Compare tuned optimizers across sizes and horizons | Match tuning effort and compare annealed endpoints |
| Delphi | Evaluate a prescribed scaling recipe on IsoFLOP slices and larger holdouts | Separate recipe heuristics from fitted loss laws |
| OLMo development | Data probes, mixture proxies, capability checks, released training artifacts | Preserve stage and data lineage |

## Reconstruct an IsoFLOP experiment

Choose a compute budget $C_i$. For candidate sizes $N_j$, choose token horizons satisfying the declared compute function; $D_{ij} \approx C_i/(6N_j)$ is a first approximation. Train the candidates under the selected tuning policy. Fit a smooth loss curve against $\log N$ within the slice and locate its minimum. Repeat across budgets, then fit how the minima scale. Inspect whether a slice minimum sits at the grid boundary before extrapolating.

## Reconstruct a fair optimizer comparison

For every size and horizon, tune each optimizer within a comparable resource budget. Compare loss after the declared decay phase, confirm finalists with fresh seeds, and report both token efficiency and wall-clock cost. An advantage that changes across sizes needs a scale-dependent model rather than a single multiplier.

## Reconstruct a recipe-prediction experiment

Freeze a function mapping compute and candidate size to architecture, batch, learning rate, schedule, and tokens. Fit on smaller budgets, commit predictions, then train stepped larger holdouts. Diagnose both loss drift and numerical failure. Revising the heuristic after observing a holdout starts a new development cycle.

This explanation is a reconstruction of methods summarized by the blog, not a claim that this repository has run these experiments. Exact numerical settings should be taken from the downloaded report and matching code revision before reproduction.

**Sources:** blog Appendix A; Chinchilla, 2022; Fantastic Optimizers, 2025, Tables 2–3; Delphi report; OLMo 3.

Blog/notes crosswalk · Detailed companion